2024 10th International Symposium on System Security, Safety, and Reliability (ISSSR) | 979-8-3503-6293-0/24/$31.00 ©2024 IEEE | DOI: 10.1109/ISSSR61934.2024.00075

Research on trustworthy Software Testing Techniques Based on Large Models

Qiang Han[1], Zijie Shi[2], and Zhenyu Zhao[2]
[1]School of Computer Scinece and Engineering, North Minzu University, Yinchuan, Ningxia, 750021, China
[2]School of Computer Science, Beijing University of Posts and Telecommunications, Beijing 100876, China
hanqiang@nmu.edu.cn, 1368066565@qq.com, yolo441300@gmail.com

Abstract—This paper focuses on trustworthy software testing techniques enabled by large language models. We first give an overview of large language models and their applications in software engineering. Then we analyze the challenges of applying these models to software testing, which include overfitting, bias and trust issues. We evaluate LMTester on both synthetic and real-world programs compared with baseline testing tools. The results demonstrate its advantages in improving test coverage, detecting bugs, and assuring trustworthiness. We also discuss the limitations of current techniques and point out several future directions. Our work is expected to facilitate trustworthy applications of large language models in software testing and quality assurance.

Keywords: ***Software Testing; Large Models; Software Trustworthiness***

1. INTRODUCTION

Software testing is an essential part of the software development life cycle, aimed at evaluating and improving the quality of software products [1,2]. With the rapid development of artificial intelligence, large language models such as GPT-3 and PaLM have shown powerful capabilities in various natural language processing tasks [3]. Leveraging the knowledge and reasoning abilities of these models, new software testing techniques are emerging to enhance the automation, intelligence, and trustworthiness of the testing process [4,5]. To address these problems, we propose a testing framework named LMTester, which integrates large language models with metamorphic testing [6], a trustworthy testing method that uses metamorphic relations of programs. Specifically, it automatically generates new test cases and oracles to validate software behaviors based on transformation rules captured by large language models.

2. ANALYZE THE CHALLENGES OF APPLYING LARGE LANGUAGE MODELS TO SOFTWARE TESTING

- Overfitting: Large language models with hundreds of billions of parameters can easily overfit to biases and spurious correlations in the training data, resulting in unpredictable behaviors when applied to new software systems [7].

- Lack of reasoning: Most models focus on predictive accuracy based on statistical patterns. They lack the ability to reason about program semantics, behaviors and specifications [8].

- Bias and fairness: Existing models reflect various social biases in the training data, leading to unfair and discriminative testing results [9].

- Transparency and explainability: The black-box nature of large neural models makes it hard to interpret testing outcomes and establish trust [10].

- Combinatorial explosion: The search space for software test data grows exponentially with program scale and complexity, posing efficiency challenges [11].

- Assuring model consistency at scale: Constant model updates under distribution shifts can lead to inconsistent testing behaviors over time [12].

We will further elaborate on these challenges in the rest of the paper. By recognizing these open problems early on, our testing methodology is designed to mitigate or avoid them appropriately.

3. PROPOSE A TESTING FRAMEWORK TO INTEGRATE LARGE LANGUAGE MODELS WITH METAMORPHIC TESTING

- Overview: We propose LMTester, a testing framework that combines the knowledge capacity of large language models and the trustworthiness of metamorphic testing.

- Architecture: LMTester consists of 1) a large language model (LLM) encoder to generate test cases, 2) a test driver to execute tests, 3) a metamorphic validator to check outputs, and 4) a consistency regulator to assure model behaviors.

- LLM encoder: Automatically encodes program semantics and generates diverse test case inputs based on the pre-trained knowledge of LLMs.

- Metamorphic validator: Validates test outcomes based on program-specific metamorphic relations learned by the LLM encoder, alleviating the oracle problem.

- Consistency regulator: Monitors model behaviors and triggers re-training if prediction inconsistency is above a threshold, ensuring robustness.

- Workflow: The test driver iterates test case execution, metamorphic output validation and consistency checking to expand the test coverage of the software under test.

We provide the technical details of each LMTester component in the following sections, including model architectures, algorithms and theoretical foundations. The framework design specifically targets the challenges mentioned previously to facilitate trustworthy LLM testing. We will analyze its effectiveness in later sections.

4. DESIGN TEST CASE GENERATION AND VALIDATION METHODS BASED ON LARGE LANGUAGE MODELS

- Test case generation
  - Encode program semantics into LLM prompt design
  - Leverage few-shot learning to adapt pre-trained LLMs
  - Generate diverse test cases via probabilistic sampling
- Validation oracle creation
  - Infer metamorphic relations from program structure
  - Formulate relations as auxiliary prompts for LLMs

2835-2823/24/$31.00 ©2024 IEEE
DOI 10.1109/ISSSR61934.2024.00075

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on August 29,2026 at 11:21:37 UTC from IEEE Xplore. Restrictions apply.

- Predict expected outputs for test cases
- Combination strategies
  - Integrate case generation and validation to enable end-to-end testing
  - Adjust sampling and thresholds to balance coverage and effectiveness
  - Iterate to expand test coverage and robustness

We provide details on prompt formulation, sampling methods, prediction aggregation, and the joint optimization of coverage and correctness. Our goal is to produce test cases to maximally cover program logic and behaviors while accurately validating outputs. We empirically compare our approach to baseline methods in test case quality, oracle correctness, and efficiency. Results will be presented in the evaluation section.

5. Conduct comprehensive evaluation on effectiveness and trustworthiness

- Test subjects
  - Synthetic programs with injected flaws
  - Real-world open-source projects
- Evaluation metrics
  - Code coverage
  - Fault detection rate
  - Test accuracy
  - Confidence intervals
  - Bias indicators
  - Prediction consistency
- Baselines
  - Traditional test case generation
  - Metamorphic testing without LM
  - Manual test suite
- Results
  - Outperform baselines on coverage and effectiveness
  - Mitigate biases and assure model consistency
  - Scale efficiently to large codebases
- Analyze limitations on complex and evolving software
  - Identify remaining challenges in real-world testing
  - Discuss trade-offs on stability vs. effectiveness

Through extensive experiments on various test subjects, we demonstrate the advantages of our LMTester over baseline approaches in testing quality, efficiency and trustworthiness. We also discuss its scope and limitations as an indication for future work.

6. Discuss limitations and future directions for research

- Limitations
  - Dealing with complex logic branches
  - Handling external dependencies
  - Updating frequently evolving software
- Future directions
  - Advanced program encoding techniques
  - Incorporating symbolic execution
  - Enhanced consistency regularization
  - Achieving complete path coverage
  - Generalizing across software domains

While showing promising results, our approach has limitations in scaling to certain complex, dynamically changing software due to fundamental challenges of LLMs. We will discuss each limitation in depth and analyze the underlying technical gaps. These insights provide potential directions for advancing reliable program analysis with large neural models. Specifically, we see integrating symbolic techniques, improving consistency over time, and testing full program paths as impactful areas needing further exploration from the research community. We hope our work and analysis could inform and stimulate future progress on applying large language models for trustworthy software testing and beyond.

The discussion sets the stage for mapping out the possibilities and challenges for lifting neural program analysis and testing to a new level of effectiveness.

References

[1] Y. Jia and M. Harman, “An analysis and survey of the development of mutation testing,” IEEE Transactions on Software Engineering, vol. 37, no. 5, pp. 649-678, 2011.

[2] C.E. Lai and C.Y. Huang, “Developing a Modified Fuzzy-GE Algorithm for Enhanced Test Suite Reduction Effectiveness,” International Journal of Performability Engineering, vol. 19, no. 4, pp. 223-233, 2023.

[3] T. B. Brown et al., “Language models are few-shot learners,” in NeurIPS, 2020.

[4] W. Ma, J. Chen, T. Liu, Y. Zhao, B. Xu, C. Chen and J. Peng, “DeepGauge: Comprehensive and multi-granularity testing criteria for gauging the robustness of deep learning systems,” in ASE, 2020.

[5] G. Yi, Z. Chen, Z. Chen, W.E. Wong, and N. Chau, “Exploring the Capability of ChatGPT in Test Generation,” In 2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security Companion (QRS-C), IEEE, pp. 72-80, 2023.

[6] T. Y. Chen, S. C. Cheung and S. Yiu, “Metamorphic testing: A new approach for generating next test cases,” Technical Report, HKUST, 1998.

[7] M. Laugel et al., "Issues in estimating model accuracy from synthetic data," in ICML, 2021.

[8] J. Lo et al. "Programming language: Challenges for AI safety," in ICLR, 2022.

[9] N. Mehrabi et al. "A survey on bias and fairness in machine learning," ACM CSUR, 2021.

[10] C. Zhang et al. "Interpretability for automatic machine learning," in ICCAD, 2021

[11] S. Anand et al., “An orchestrated survey of methodologies for automated software test case generation,” Journal of Systems and Software, vol. 86, no. 8, pp. 1978-2001, 2013.

[12] L. Deng, “The MNIST Handwritten Digit Database, Yann LeCun, Corinna Cortes and Chris Burges,” URL: http://yann.lecun.com/exdb/mnist/, 1998.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on August 29,2026 at 11:21:37 UTC from IEEE Xplore. Restrictions apply.